

CSEN 296B-2 | Week 2.1 Lab Submission

Working Agreement for AstraNotes

Student Name: Alex Shirazi

Date: April 6, 2026

Project: AstraNotes

Chosen Technical Path: Python

1. Working Agreement for AstraNotes

This Working Agreement explains how I will manage my AstraNotes project as an AI-native individual workflow over the quarter.

1.1 How I Will Plan and Track Work

All AstraNotes work is organized around a lightweight sprint structure aligned to the course week boundaries. At the start of each week, I review the current milestone deliverable and break it into discrete tasks that map directly to one or more requirements in the Requirement Set (FR, NFR, or SPR series from Submission 2). Each task is written as a user story: "As a user, I want [capability] so that [value]."

- Tasks are tracked in a backlog.md file in the project repository, each carrying a status (Backlog / In Progress / Done) and a linked requirement ID (e.g., FR-01, SPR-03).
- No task enters In Progress without a linked requirement — this prevents scope creep driven by AI suggestions.
- At the end of each sprint I log two questions: what slowed me down this week, and what would I do differently next sprint.

1.2 How I Will Use AI Inside My Workflow

AI assistance is treated as a structured collaborator, not an autocomplete engine.

- Frame the task first. Before invoking any AI tool, I write a one-sentence problem statement and identify which requirement(s) the task addresses.
- Use a tiered prompt strategy: weak baseline, then a structured strong prompt with explicit constraints (context, architecture, edge cases), then a refinement prompt targeting gaps.
- Scope AI to generation, not decision-making. AI generates code and documentation drafts; I decide what to use, why, and how it integrates with the existing codebase.
- Constrain output to project boundaries. Suggestions that bypass the NoteRepository interface, PrivacyService, or ValidationLayer are rejected or parked as ideas.

1.3 How I Will Document Prompts, Decisions, and Revisions

Every significant AI interaction is logged in `ai_log.md` in the project repository. Each entry records the date, the backlog task it supports, the full prompt used, a 2–3 sentence summary of the response, the decision made (accepted / rejected / modified and why), and the requirement IDs the work satisfies.

Architecture-level decisions are additionally maintained in the Architecture Decision Log from Submission 1. The ADL captures design rationale; `ai_log.md` captures AI interaction history.

1.4 How I Will Decide Whether AI Output Is Acceptable

I will not move AI-generated output forward unless it passes all of the following:

- Requirement alignment: the output traces to at least one identified requirement; generic output is not used as-is.
- Architectural fit: the output respects layer boundaries and does not introduce undocumented dependencies.
- Explainability: I can walk through every line or paragraph in my own words before integrating it.
- Privacy and security compliance: any output touching note data routes private notes through `PrivacyService` and does not expose note content in error messages.
- Reasonableness: the approach matches how a careful Python developer would solve the problem; overly complex suggestions are re-prompted for a simpler alternative.

1.5 How I Will Prevent Drift, Duplication, and Low-Quality Output

- Drift is managed by re-reading the relevant requirement before generating new output. If new AI output contradicts or silently expands an existing design decision, the conflict is resolved in the ADL before any code is written.
- Duplication is prevented by checking the existing codebase before generating a new module or function. AI prompts explicitly instruct the model not to re-implement existing components.
- Low-quality output is addressed through prompt discipline and the acceptability checks above. A weekly self-review of `ai_log.md` flags recurring quality problems so I can adjust my prompting strategy.

2. Short Reflection

Building this Working Agreement against the actual AstraNotes project revealed a gap I had not consciously identified before: I had no formal rule about when to stop iterating on an AI response and simply write the solution myself. Going forward I will apply a two-strike heuristic

— if a second refined prompt still does not produce acceptable output, I write the component manually and log the failed attempt as a learning record.

The Working Agreement also forces me to treat my requirement set as a live contract rather than a one-time submission. Every AI-assisted task now starts with a requirement ID lookup. If I cannot find a requirement for a task I want to do, that is a signal to either add it formally or reconsider whether the task belongs in the project at all. Over the quarter, this discipline will prevent the accumulation of unmapped, unreviewed features that weaken the project's coherence.